

Maintenance Kiosk Test Plan

A complete verification plan: every function, its edge cases, and its reliability under real workshop conditions. Print it, run a column of ticks down the Result cells, and you know exactly what state the kiosk is in.

118 manual cases

52 automated tests

Laptop + Raspberry Pi

September 2026 • v1

1 Strategy & criteria

Three layers of testing, each catching what the layer below cannot.

- **Layer 1 – automated regression (52 tests).** Runs in ~1 second, catches broken logic. Run on every code change, before every commit, and before every Pi deploy. It is the floor, never the whole answer.
- **Layer 2 – manual functional tests (section 4-5).** A human clicking through every screen and form the way the team will, including the wrong ways. Run the full catalog before a semester launch and after big changes; run the smoke subset (section 7) before every deploy.
- **Layer 3 – reliability & resilience (section 6).** Power cuts, dead network, crashes mid-save, weeks of uptime. Run once on the Pi before go-live and again each semester.

Priorities

PRIORITY	MEANING	RULE
P1	Core workflow or data safety – the kiosk is not usable/trustworthy without it	Every P1 must pass before any deploy. A P1 failure stops the release.
P2	Important behavior with a workaround	Failures triaged; fix before semester launch.
P3	Cosmetic, convenience, rare edge	Log it; fix opportunistically.

Entry / exit criteria

- **Entry** (before manual testing starts): automated suite green; app boots with a fresh database; test Discord channel and test spreadsheet ready.

- **Exit** (to call the build good): 100% of P1 pass · no open Critical/Major bugs · every P2 failure has a written decision (fix now / accept) · results sheet dated and kept.

2 Environments & test data

Never test against the team's real Discord channel or the live database.

ENVIRONMENT	WHAT IT'S FOR	NOTES
Laptop dev	Layers 1-2, fast iteration	<code>.venv/bin/python app.py</code> → <code>http://127.0.0.1:5050</code> · fresh SQLite DB
Pi staging run	Layer 2 re-check + all of Layer 3	The real Pi, real touchscreen, systemd services installed – before it goes on the wall

Test fixtures (set up once)

- **Fresh database:** stop the app, delete `data/kiosk.db`, start again – demo printers reseed automatically. Do this to start every full pass from a known state.
- **Test Discord webhook:** make a private channel `#kiosk-testing` with its own webhook; paste that into Settings while testing. Swap the real one in only at go-live.
- **Test spreadsheet:** a separate Google Sheet + Apps Script deployment for sync tests.
- **Test technician:** add yourself with a real email you can check, reminder box ticked.
- **Known tree state:** before editor tests, note the node count shown in the editor header so you can confirm restore later.

DATA SAFETY RULE

Once the kiosk is live, never run destructive tests (fresh-database resets, restore drills, editor stress tests) on the production Pi database. Back up first (Shipping Manual §7), or run them on the laptop.

3 Automated regression suite

52 tests · ~1 second · run it constantly.

```
.venv/bin/python -m unittest discover tests      # expect: Ran 52 tests ... OK
```

FILE	TESTS	COVERS
test_backend_persistence.py	7	SQLite fallback, weekly save persistence, dashboard paging, SOP viewer + assets, diagnosis Back links, manual reactive save
test_settings_notifications.py	18	Technicians, settings validation, Discord + Google Chat queue/payloads, worker success/failure/retry, weekly reminder targeting, worker bootstrap
test_settings_audit_admin.py	23	Log diary write/filter/render, typed settings fallbacks, printer fleet edit, semesters, sheets settings, PIN gating + logging + validation, tree validation, editor round-trips, backup/restore, delete-while-referenced
test_google_sheets_sync.py	4	Payload format, failure never blocks a save, retry worker, backfill

THE STANDING RULE

Every bug found by manual testing gets an automated test when it is fixed, so it can never come back silently. The suite only grows.

4 Functional test catalog

Every screen, every form, every button – including the wrong inputs.

Work module by module on a fresh database. "→" means "then". Write P (pass), F (fail), or S (skipped) in the Result column, with the date at the top of the sheet.

A • Dashboard & navigation (DASH)

ID	TEST	EXPECTED	P	RESULT
DASH-01	Open <code>/</code> on a fresh DB	Dashboard renders, demo printers in 2×2 cards, group buttons + Previous/Next	P1	
DASH-02	Page through all groups (Mainspace / Digital / Exotic / Next / Previous)	All 10 active printers reachable; current group highlighted; Previous disabled on first page	P2	
DASH-03	Leave the dashboard open 1 minute untouched	Auto-refreshes at the interval set in Settings; no flicker to error, clock ticks	P1	
DASH-04	Trigger a fault on printer X from another browser → watch the dashboard	X's card turns red with fault text within one refresh interval, header fault count +1	P1	
DASH-05	Card colour/chip agreement: make one printer Available, one Under Maintenance, one Unavailable	Border colour, chip text, and chip colour agree on every card (green/amber/red)	P2	
DASH-06	Nozzle-life bar: record filament use near the configured limit	Progress bar fills proportionally; ≥ limit is visually flagged	P3	
DASH-07	GET <code>/api/dashboard?page=2</code> directly	Valid JSON for page 2; <code>page=0</code> or <code>page=99</code> returns a sane page, not a crash	P3	
DASH-08	Open <code>/printers/999</code> (no such printer)	Clean 404 – not a stack trace	P2	

B • Printer page & Quick Fix (PRNT)

ID	TEST	EXPECTED	P	RESULT
PRNT-01	Tap a dashboard card	Printer page shows correct name, status, dates, last fix; Dashboard link returns	P1	
PRNT-02	All six action buttons	Weekly Maintenance / Reactive Start / Weekly History / Full History / Assist / Manual Log each open the right page for the right printer	P1	
PRNT-03	Tap a Quick Fix (e.g. "Nozzle clog fixed")	Save Outcome opens pre-filled for that fix; saving writes history + updates card	P1	
PRNT-04	Each Diagnosis SOP card on the printer page	Opens the assistant already on that symptom's first question	P2	
PRNT-05	Recent history panel after 2-3 saves	Shows the latest events, newest first; links open the right record	P2	

C • Weekly maintenance (WKLY)

ID	TEST	EXPECTED	P	RESULT
WKLY-01	Fill the weekly form completely and save	Redirects to Weekly History with the record listed; dashboard "WEEKLY" date updates to today	P1	
WKLY-02	Semester and week dropdowns	Semesters match Settings; weeks run 1 to the configured weeks-per-semester	P2	
WKLY-03	Technician dropdown	Lists exactly the active technicians from Settings	P2	
WKLY-04	Save with only some checklist boxes ticked	Record stores exactly the ticked/unticked state; detail view shows it faithfully	P1	
WKLY-05	Metric fields: leave blank / enter 0 / enter a decimal	All accepted and displayed back correctly; no crash on blank	P2	
WKLY-06	Weekly History → open a record	Read-only detail matches what was saved, field for field	P1	
WKLY-07	Save two records for the same printer, same week	Both kept in history (or a defined overwrite rule) – no silent data loss either way	P2	
WKLY-08	Refresh the browser on the redirect page after saving	No duplicate record created by the refresh (POST-redirect-GET holds)	P2	

D • Reactive fault workflow (REAC)

ID	TEST	EXPECTED	P	RESULT
REAC-01	Reactive Start → fill origin/symptom/urgency → Manual	Fault event created "in progress"; dashboard card immediately red/amber with the fault text	P1	
REAC-02	Reactive Start → Diagnosis path	Assistant opens on the chosen symptom category	P1	
REAC-03	Save Outcome → Available	Fault closes, card green, history row written with technician + fix summary, "fault fixed" notification queued	P1	
REAC-04	Save Outcome → In Progress	Card amber "Under Maintenance", fault stays open, history row written	P1	
REAC-05	Save Outcome → Unavailable	Card stays red, fault open, history row written	P1	
REAC-06	With a fault open, enter Manual Log again on the same printer	Header shows it continues the existing event – no duplicate fault created	P1	
REAC-07	Fix the fault, then start a new one on the same printer	A fresh event starts; Full History shows both, clearly separated	P2	
REAC-08	Save Outcome with technician left on "Select"	Defined behavior: either blocked with a message, or saved with blank technician – but consistent and visible in history	P2	
REAC-09	Full History after several mixed events	All weekly + reactive events listed newest-first; filter links work	P1	
REAC-10	Very long issue text (500+ characters, emoji, quotes)	Saves, displays without breaking layout, appears truncated-but-safe on the dashboard card	P3	

E • Diagnosis assistant (DIAG)

ID	TEST	EXPECTED	P	RESULT
DIAG-01	Assist → symptom screen	One card per category, in display order, matching the editor	P1	
DIAG-02	Walk one full path to a diagnosis	Every answer leads somewhere; breadcrumb grows with each answer; result shows causes/actions/SOPs	P1	

DIAG-03	Back from a deep question, twice	Steps back one question at a time with breadcrumb shrinking correctly; from the first question, Back returns to symptom picker	P1	
DIAG-04	Accept Diagnosis	Save Outcome opens pre-filled with the node's fault, symptom, component, action and fix summary	P1	
DIAG-05	SOP links on a result page	Each opens the right SOP; a result with no SOP shows "No linked SOP", not an error	P2	
DIAG-06	Exhaustive branch walk – every answer of every question of every category (use the editor listing as the checklist)	No dead end, no wrong destination, no question whose answers all loop; every leaf reachable	P1	
DIAG-07	URL tampering: <code>?category=nope</code> and <code>&node=nope</code>	Clean 404 for both – never a stack trace	P2	
DIAG-08	Edit a question's wording in the editor → reload the assistant	New wording appears immediately, no restart needed	P1	

F • Diagnosis editor (EDIT)

ID	TEST	EXPECTED	P	RESULT
EDIT-01	Open <code>/admin/diagnosis</code>	All categories with their questions/diagnoses grouped correctly; node count in header; no error banners on a healthy tree	P1	
EDIT-02	Edit a question's text + help text → save	Saved, banner confirms, assistant shows it (DIAG-08)	P1	
EDIT-03	Repoint an answer's "Goes to" → save → walk that path	The answer now leads to the new node	P1	
EDIT-04	Add an answer option (label + destination); remove one by blanking its label	Both changes stick; the assistant shows the new button set	P1	
EDIT-05	Save a question with zero options, or an option with no destination	Save is <i>blocked</i> with a plain-English error; nothing written	P1	
EDIT-06	Create a new diagnosis, then wire an answer to it, then walk to it	Full loop works; generated id starts <code>d_</code> ; Save Outcome prefill uses its fields	P1	
EDIT-07	Create a new question + new category using it as start node	New symptom card appears on the assistant, opens the new question	P1	

EDIT-08	Delete a node that is still referenced	Refused, message lists exactly who references it	P1	
EDIT-09	Repoint the last reference away, then delete the node	Delete succeeds; editor shows no orphan errors	P2	
EDIT-10	Delete a category	Symptom card gone; its nodes remain and show as "unreachable" warnings (not errors)	P2	
EDIT-11	Raw JSON: paste invalid JSON; then valid JSON with a dangling <code>next</code>	Both rejected with the reason; the tree file is untouched (re-open to confirm)	P1	
EDIT-12	Raw JSON: a valid edit	Saves; assistant reflects it; a new backup appears in the restore list	P2	
EDIT-13	Restore the backup from before EDIT-02	Tree returns to the noted earlier state (node texts back); restore is itself logged in the log diary	P1	
EDIT-14	Category form: display order non-number; blank label	Both rejected with clear messages	P3	

G • Settings (SETT)

ID	TEST	EXPECTED	P	RESULT
SETT-01	Add two technicians (one reminder-ticked), save, reload	Both persist; ticked state persists; both appear in form dropdowns	P1	
SETT-02	Technician with invalid email	400 with "valid email" message; nothing saved	P1	
SETT-03	Rename a printer; reorder two printers; add an 11th; untick Active on one	Dashboard follows all four changes; retired printer's history still opens	P1	
SETT-04	Two printers with the same name	Rejected with a clear message	P2	
SETT-05	Webhook fields with <code>http://</code> (not https)	Rejected for Discord and Google Chat alike	P1	
SETT-06	Nozzle life, weeks-per-semester, page size, refresh seconds, retention: save valid values → check each takes effect; then try 0 / negative / text	Valid values change behavior (dashboard label, week dropdown, paging, refresh, log prune); invalid values rejected with 400	P1	
SETT-07	Edit semester labels/dates; add the optional third semester	Weekly form dropdown follows	P2	
SETT-08	Kiosk base URL round-trips and appears in the next reminder email's link	Saved value used in email body (verify with NOTF-06)	P2	
SETT-09	Sheets sync: tick + URL + secret round-trip; secret shows as "set" not plaintext	Saved; sync module reads the DB values (SHTS tests)	P2	

H • Admin PIN & access control (AUTH)

ID	TEST	EXPECTED	P	RESULT
AUTH-01	Fresh DB: open Settings and the editor with no PIN set	Both open; yellow "no PIN" banner shows on Settings	P2	
AUTH-02	Set a PIN → open <code>/settings</code> and <code>/admin/diagnosis</code> in a fresh browser	Both redirect to the PIN screen; correct PIN opens the page you asked for (next-URL preserved)	P1	
AUTH-03	Wrong PIN, three times	"Wrong PIN" each time, no lockout surprise, each attempt logged as a warning in <code>/logs</code>	P1	
AUTH-04	PIN form: mismatch in confirm field; letters; 3 digits	All rejected (4-8 digits required)	P2	
AUTH-05	Set session length to 1 minute → unlock → wait 2 minutes → revisit Settings	PIN asked again (session expired)	P2	
AUTH-06	<code>/admin/logout</code> after unlocking	Next admin page visit asks for the PIN again	P2	
AUTH-07	With PIN set: normal kiosk pages (dashboard, forms, assistant, <code>/logs</code>)	All still open with no PIN – only Settings + editor are gated	P1	
AUTH-08	Login <code>next</code> tampering: <code>/admin/login?next=https://evil.example</code>	After PIN, lands on a kiosk page (Settings), never an external site	P2	

I • Log diary (LOGS)

ID	TEST	EXPECTED	P	RESULT
LOGS-01	Do a weekly save, a fault save, a settings change, a tree edit → open <code>/logs</code>	All four actions appear, newest first, with sensible descriptions	P1	
LOGS-02	Filter by level (error/warning) and by source; search box	Filters combine correctly; empty result shows an empty state, not an error	P2	
LOGS-03	Generate 60+ entries → pagination	50 per page, page controls work, totals correct	P3	
LOGS-04	Cause a failure (e.g. worker with a bad webhook) → check <code>/logs</code>	The failure is recorded as an error entry with the reason	P1	

J • SOP viewer (SOPS)

ID	TEST	EXPECTED	P	RESULT
SOPS-01	Open every SOP folder in <code>sops/</code> via <code>/sops/<name></code>	Each renders with its styling; all images load (no broken image icons)	P1	
SOPS-02	<code>/sops/does-not-exist</code>	Clean 404	P2	
SOPS-03	Asset path traversal: <code>/sop-assets/<slug>/../..../.env</code>	Refused (404/400) – must not serve files outside the SOP folder	P1	
SOPS-04	SOP links from a diagnosis result and from the tree's SOP list	Titles/descriptions match the tree's <code>sops</code> section; links resolve	P2	

5 Integration tests — the outside world

Real Discord, real email, real Google Sheets. Use the test channel/sheet.

K • Notifications (NOTF)

ID	TEST	EXPECTED	P	RESULT
NOTF-01	Discord webhook set → trigger a fault → run <code>notification_worker.py</code>	"Fault started on ... " message in the test channel; queue row marked sent	P1	
NOTF-02	Mark the fault fixed → run worker	"Fault fixed ..." message with technician and fix summary	P1	
NOTF-03	No webhooks configured → trigger a fault	Save succeeds normally; no queue rows for chat channels; nothing errors	P1	
NOTF-04	Both Discord and Google Chat configured → one fault	Both channels receive the message (one queue row each)	P2	
NOTF-05	Deliberately break the webhook URL (garbage path) → worker	Worker reports failure; row status "failed", retry_count grows; save was never affected; error visible in <code>/logs</code>	P1	
NOTF-06	Run <code>weekly_reminder_worker.py</code> then <code>notification_worker.py</code>	Reminder email arrives only for ticked technicians; link uses the Settings base URL	P1	
NOTF-07	Fault with a very long issue text (>2000 chars)	Discord message arrives truncated at Discord's limit – not rejected	P3	
NOTF-08	Fix the broken webhook from NOTF-05 → run worker again	The failed rows send on retry and mark sent – nothing lost	P1	

L • Google Sheets sync (SHTS)

ID	TEST	EXPECTED	P	RESULT
SHTS-01	Sync enabled + test sheet → weekly save	Weekly row + printer status row appear in the sheet (immediately or after the worker)	P1	
SHTS-02	Reactive save (started, then fixed)	Reactive log rows + status updates appear	P1	

SHTS-03	Sync disabled → saves	No calls to Google; saves unaffected; rows wait in <code>sync_queue</code>	P2	
SHTS-04	Wrong secret → save → worker	Save fine; sync marked failed with reason; fixing the secret + rerun delivers the backlog	P1	
SHTS-05	<code>--backfill</code> on a DB with existing history	Historic records queued and delivered once – run twice, no duplicate flood	P2	

6 Reliability & resilience

Run on the real Pi with systemd installed. This is what "reliable" means.

ID	TEST	EXPECTED	P	RESULT
REL-01	Crash recovery: <code>sudo systemctl kill -s KILL maintenance-kiosk</code> while the kiosk is idle	systemd restarts it within ~5 s; dashboard recovers on next refresh without touching the Pi	P1	
REL-02	Crash mid-workflow: kill the service while a Save Outcome form is filled but not submitted, then submit after restart	Either the save lands cleanly or a clear error appears – never a half-written history row (check Full History + <code>/logs</code>)	P1	
REL-03	Power cut: pull the Pi's plug, wait 10 s, power on	Boots to the full-screen dashboard unattended; services + all 3 timers active; database intact (spot-check history)	P1	
REL-04	Network down: unplug Ethernet/Wi-Fi → trigger a fault + weekly save → wait 10 min → reconnect	All saves succeed offline; Discord/Sheets messages queue with retries; everything delivers within ~5 min of reconnection, nothing lost, nothing duplicated	P1	
REL-05	Missed Monday: Pi off across Monday 12:00 → boot Tuesday	Persistent timer fires the reminder once at boot – exactly once (check email + queue)	P1	
REL-06	Concurrency: kiosk touchscreen + two laptop browsers saving on different printers within seconds	All saves land; no SQLite "database is locked" errors surfacing to users (check <code>/logs</code> + <code>journalctl</code>)	P2	
REL-07	Double-tap: rapid double-tap on Save buttons (touchscreens do this)	One record, not two – for weekly, outcome, and settings saves	P1	
REL-08	Soak: leave the kiosk running 7 days untouched	Screen never blanks, browser never white-screens, memory stable (<code>free -h</code> day 1 vs day 7), auto-refresh still live, clock correct	P1	
REL-09	Tree backup pruning: save 27 editor edits in a row	Backup list holds the newest 25 only; restore of the newest works (laptop test)	P3	
REL-10	Backup drill: follow Shipping Manual §7 backup, then restore the copy on a laptop clone	Restored app shows full history and settings – proves the backup actually restores	P1	
REL-11	Update drill: commit a trivial change → <code>git pull</code> + restart on the Pi per Shipping Manual §7	New code live; database, settings, and tree edits all preserved	P1	
REL-12	Log retention: set retention low (e.g. 1 day) with old entries present	Old entries pruned per setting; recent kept; no runaway table growth	P3	

LOOK OUT FOR DURING THE SOAK

The three quiet failure modes on a Pi kiosk: SD-card corruption (mitigated by the backup drill), Chromium slowly eating memory (caught by the day-7 check), and mDNS (`raspberrypi.local`) failing while the IP still works (test both during REL-08).

7 Suites — what to run when

The full catalog is for launches. Day to day, run the right subset.

SUITE	WHEN	CONTENTS	TIME
Commit check	Every code change	Automated suite (unittest discover)	~1 s
Smoke	Before every deploy; after any config change	DASH-01, DASH-04, PRNT-02, WKLY-01, REAC-01, REAC-03, DIAG-02, DIAG-04, EDIT-02, AUTH-02, NOTF-01, SHTS-01	~15 min
Full functional	Semester start; after large features	All of sections 4-5 (fresh DB first)	~half a day
Reliability drill	Before go-live; once per semester	All REL cases (REL-08 runs in the background for a week)	~2 h active + 7-day soak

DEFINITION OF "READY FOR THE WALL"

Automated suite green on the Pi + smoke suite passed on the Pi + REL-01, REL-03, REL-04 and REL-10 passed. Everything else can follow while the kiosk is already in use.

8 Recording results & reporting bugs

A test that isn't written down didn't happen.

Results

Print this document (the Result column is there to be written in), or duplicate the tables into a spreadsheet. Head each run with: date, tester, environment (laptop / Pi), git commit (`git log --oneline -1`). Keep completed sheets in `docs/` or the team drive – they are the kiosk's health record.

Bug report template

```
ID:          BUG-YYYYMMDD-n
Found by:    test case ID (or "exploratory")
Env:         laptop / Pi · git commit · date
Severity:    Critical (data loss / kiosk down) | Major (P1 feature broken)
              | Minor (workaround exists) | Cosmetic
Steps:       1. ... 2. ... 3. ...
Expected:    ...
Actual:      ... (screenshot if visual)
Evidence:    /logs entry, journalctl excerpt, or worker output
```

- **Critical** bugs stop testing until understood – especially anything touching saved history.
- When a bug is fixed, add an automated test for it (section 3's standing rule), re-run the failing case, and only then mark it closed.

Maintenance Kiosk Test Plan · v1 · September 2026 · companion to the Team Manual and the Pi Shipping Manual · automated suite: `tests/` (52 tests).